



Universidad Central de Venezuela
Facultad de Ciencias
Escuela de Computación
Algoritmos y Programación (6201)
Semestre (1 -2025)



Proyecto #2
SIMULADOR DE TRÁFICO URBANO

Las calles de Caracas están altamente congestionadas! Los conductores no saben cómo evitar el tráfico, las colas en la autopista son interminables y el transporte público no da abasto para tanta afluencia de personas. El *Instituto Computista de Tránsito Terrestre* (ICTT) solicita la ayuda de programadores expertos para crear simulaciones en pequeña escala, que ayuden a entender mejor el problema de tráfico en la ciudad.

En este proyecto deberás programar en C++ un simulador de tráfico que modele el flujo de vehículos en sectores de Caracas, representados en una matriz bidimensional. La simulación avanzará en iteraciones de tiempo (segundos), manteniendo un reloj global. Cada celda representará calles, cruces semaforizados o edificaciones, y los vehículos se desplazarán paso a paso según reglas de movimiento, semáforos y posibles colisiones. El programa leerá archivos de entrada para configurar el escenario y la lista de vehículos, ejecutará la simulación con cambios de estado en cada iteración y generará archivos de salida con el registro de eventos y estadísticas.

Representación del mapa

- La ciudad se representa con coordenadas X e Y entre 0 y N-1 (filas y columnas).
- Cada celda puede ser:
 - **C** (Calle): puede ser horizontal o vertical.
 - **S** (Semáforo): cambia de fase en función del reloj global.
 - **E** (Edificio): bloquea el paso.

Definición de archivos de entrada

1. mapa.in

- Primera línea: N M K (número de filas y columnas, número de segundos de la simulación).
- Siguiendo N líneas: M caracteres (C, S, E, A, M) separados por espacios.

2. config_semaforos.in

- Cada línea: $i \ j \ N \ O \ R1 \ R2 \ \dots \ Rn$ donde (i,j) es la posición del semáforo, donde N serán cuántos elementos tiene el patrón, cada Ri será el patrón que dictamina que luz está encendida en el semáforo 0 para rojo y 1 para verde y O es el inicio del patrón del semáforo.

Elementos del mapa

- **C** simboliza espacios de carretera por donde los vehículos pueden circular
- **S** simboliza los semáforos, casillas por donde los vehículos pueden pasar solo si estan en verde
- **E** simboliza los edificios, casillas por donde los vehículos no pueden pasar
- **A** simboliza los Autos, son elementos dentro de la simulación que solo se mueven de izquierda a derecha, siempre hacia la izquierda hasta que se encuentren un edificio, otro Auto o un borde del mapa, si se dan alguno de esos casos, cambia de dirección hacia la derecha.
- **M** simboliza las Motos, son elementos dentro de la simulación que solo se mueven de arriba hacia abajo, siempre hacia arriba hasta que se encuentren un edificio, otra Moto o un borde del mapa, si se dan alguno de esos casos, cambia de dirección hacia abajo.

Reglas de la simulación

- **Iteraciones de tiempo:**
 - La simulación avanza en pasos de 1 segundo según un reloj global.
- **Semáforos y sincronización:**
 - Cada semáforo tiene un ciclo igual a la longitud N de su patrón segundos.
- **Desplazamiento y esperas:**
 - Un vehículo puede moverse en una casilla si la celda está libre y no hay semáforo en rojo.
 - El orden en el que se mueven los vehículos y los semáforos siguen su patrón es el orden corresponde a cuál cada vehículo fue leído primero de la matriz de entrada, de izquierda a derecha.
 - Si debe detenerse, acumula tiempo de espera.
- **Colisiones y bloqueos:**
 - Si un auto y/o una moto intentan entrar a la misma celda mientras transcurre la simulación y alguno de ellos no puede entrar a la celda, se registra un choque o colisión y ambos vehículos pierden la capacidad de moverse por el resto de la simulación.
 - Si un vehículo está detenido por un semáforo en rojo, los vehículos que estén directamente en la casilla anterior no cambiarán de dirección.

- **Finalización:**

La simulación termina cuando han transcurrido T segundos.

Salida del programa

1. **log_simulacion.out:**

Deberá tener el estado final de la simulación

2. **estadisticas.out** con encabezado:

- Tendrá **C + M** vehiculos, donde cada línea tendrá el formato:
- ID V Tv Td Cd A , donde cada uno significa:
 - ID número de vehículo, el orden corresponde a cuál vehículo fue leído primero de la matriz de entrada, de izquierda a derecha.
 - V tipo de vehículo.
 - Tv número de segundos que paso el vehículo en movimiento.
 - Td número de segundos que pasó detenido el vehículo.
 - Cd número de cambios de dirección que tuvo que sufrir el vehículo durante la simulación.
 - A si el vehículo sufre de un choque en la simulación, será el segundo donde se produjo el choque, si no chocó, deberá ser 0.

Consideraciones

- Para el **log_simulacion.out**, si un auto esta en la casilla de un semáforo, debe imprimirse es el semáforo en el archivo
- Debe realizar un diagrama de clases en el cual se vea reflejado como uso el paradigma de programación orientada a objetos en su proyecto, el mismo debe ser entregado junto con su código fuente.
- Su solución debe tener el paradigma de programación orientada a objetos
- Que el semáforo deje pasar a los vehículos significa que lo deje entrar a la casilla donde está ubicado, una vez en ella, puede salir cuando quiera
- Todos los archivos de entrada se pasan por parámetros, al ejecutar, su programa debe ser capaz de leer la ruta completa al archivo y se hará de la siguiente manera:

`./a.exe "C:/Ruta/Al/Archivo/mapa.in" "C:/Ruta/Al/Archivo/config_semaforos.in"`

o en su defecto

`./a.exe "C:\Ruta\Al\Archivo\mapa.in" "C:\Ruta\Al\Archivo\config_semaforos.in"`

Casos de Prueba

mapa.in	config_semaforos.in	log_simulacion.out	estadisticas.out
3 3 4 C C C A S C C C C	1 1 2 2 1 0	C C C C S A C C C	1 Auto 2 2 2 0
3 3 4 C C C A C A M C C		C C C A C A M C C	1 Auto 0 4 1 1 2 Auto 2 2 2 0 3 Moto 0 4 0 1

Condiciones de entrega , restricciones y evaluación:

- Toda lógica, resolución de problema u operación debe ser implementada por el alumno, no se acepta utilización de librerías externas, exceptuando: cstdlib, stdlib, iostream, math.h, csmath o alguna libreria para el manejo de archivos.
- Se puede utilizar solamente el contenido visto en la materia.
- Todo el código debe ser entregado en un sólo archivo cpp.
- El proyecto se puede realizar sólo con alumnos de la misma sección (individual o en pareja).
- El código debe ser entregado en un zip. con el siguiente formato:

[AyP-Proyecto2][C1]-PEDRO-PEREZ-22333444_JUAN-RAMIREZ-33444555.cpp

Este archivo comprimido debe ser enviado a la dirección de correo: aypucv@gmail.com y ranaldoraffaele@gmail.com.

El asunto debe replicar el nombre del archivo, de no cumplir el formato pedido su proyecto no será corregido. destacar que C1 es una sección de ejemplo donde debe indicar la sección de los participantes.

EJEMPLO: **[AyP-Proyecto2][C1]-PEDRO-PEREZ-22333444_JUAN-RAMIREZ-33444555**

- La fecha de entrega de proyecto estipulada es: **18/07/2025 11:59pm.**

- Las entregas serán evaluadas con la versión **g++ (MinGW.org GCC-6.3.0-1) 6.3.0 o superior**, si la entrega no compila o tiene errores, el proyecto se otorgará nota mínima.
- La entrega debe ser capaz de manejar las entradas y salidas especificadas en el enunciado. Si su proyecto no es capaz de realizar el manejo de archivos, no podrá ser corregido, adicionalmente, si sus salidas no coinciden exactamente con las publicadas con el grupo docente, se tomarán como erróneas.
- Los casos de evaluación, no necesariamente contendrán los incluidos en el enunciado.
- **En caso de detección de copia y/o utilización de herramientas de IA, se le asignará la calificación mínima, sin posibilidad de apelación, además de sanciones adicionales y perdiendo la oportunidad de recuperación o reparación de la materia.**